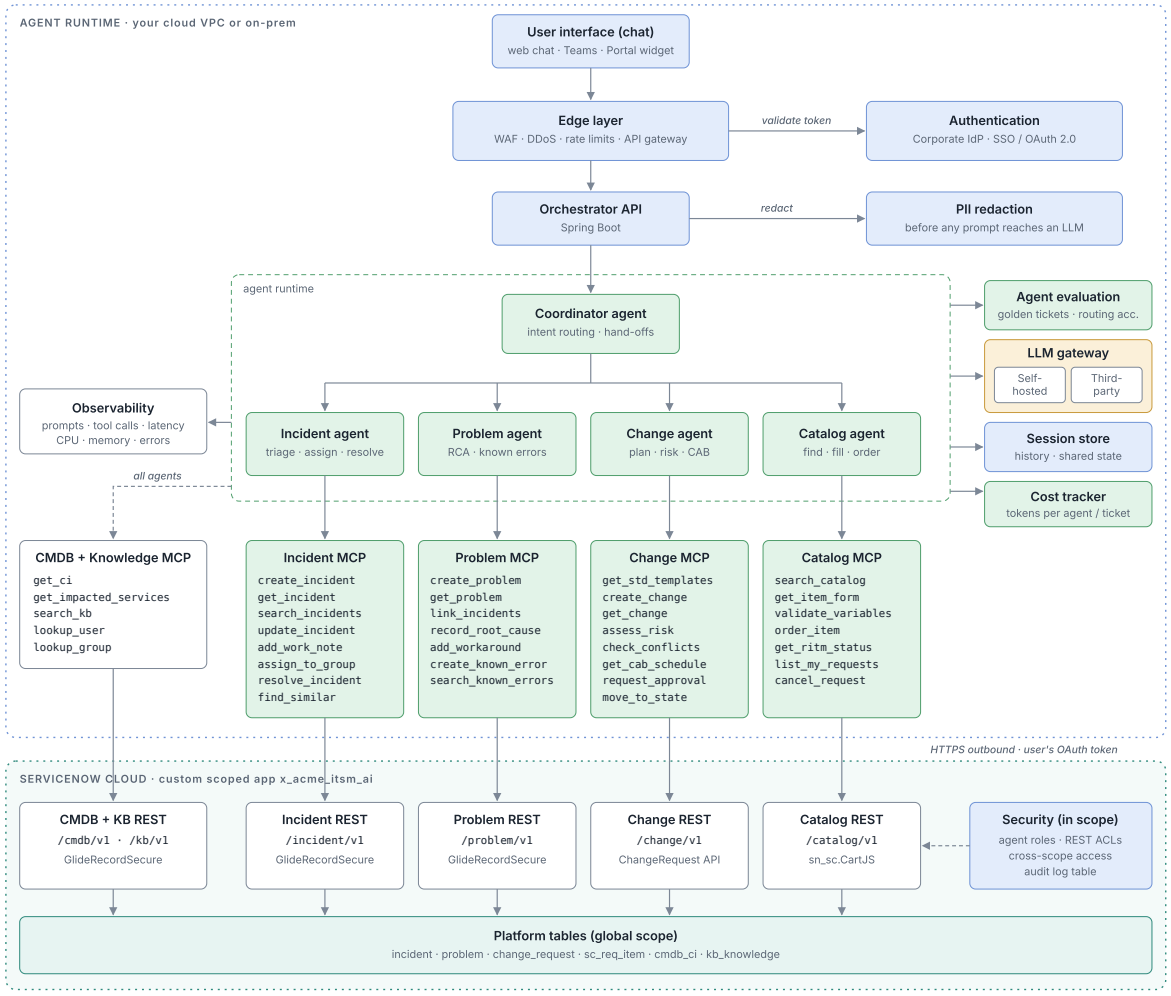
AGENTIC ITSM · REFERENCE ARCHITECTURE

ServiceNow ITSM Agent Mesh

A coordinator routes each request to one of four domain agents. Every agent owns exactly one MCP server, and every MCP server calls its own Scripted REST API in a custom scoped app, always as the signed-in user.



The agent stack runs outside ServiceNow; the only thing it can reach is the scoped app's Scripted REST APIs. All authorization, Glide queries and auditing live inside the scoped app on the ServiceNow cloud.

MCP tool catalogue

Incident MCP server

`/api/x_acme_itsm_ai/incident/v1 · GlideRecordSecure on incident`

TOOL	WHAT IT DOES	TYPE
`create_incident`	Short description, caller, CI, impact, urgency	confirm
`get_incident`	Fetch by number (INC…) or sys_id	read
`search_incidents`	Encoded query: caller, state, group, CI, date range	read
`update_incident`	Change state, priority, category	confirm
`add_work_note`	Work note or customer-visible comment	confirm
`assign_to_group`	Set assignment group / assignee	confirm
`resolve_incident`	Resolution code + close notes	confirm
`find_similar`	Recent incidents on the same CI or text match	read

Problem MCP server

`/api/x_acme_itsm_ai/problem/v1 · GlideRecordSecure on problem, kb_knowledge`

TOOL	WHAT IT DOES	TYPE
`create_problem`	Open a problem from one or more incidents	confirm
`get_problem`	Fetch by PRB number with linked incidents	read
`link_incidents`	Set `problem_id` on incidents	confirm
`record_root_cause`	Cause notes + cause CI	confirm
`add_workaround`	Workaround text on the problem	confirm
`create_known_error`	Mark known error, publish KB article	confirm
`search_known_errors`	Match symptoms against known errors	read

Change MCP server

`/api/x_acme_itsm_ai/change/v1 · ChangeRequest script API`

TOOL	WHAT IT DOES	TYPE
`get_std_templates`	List standard change templates	read
`create_change`	Normal, standard or emergency change	confirm
`get_change`	Fetch CHG with tasks and approvals	read
`assess_risk`	Run risk assessment / calculate risk	read
`check_conflicts`	Blackout windows and CI schedule conflicts	read
`get_cab_schedule`	Upcoming CAB meetings and agenda	read
`request_approval`	Move to Assess and trigger approvals	confirm
`move_to_state`	Implement, review, close	confirm

Catalog MCP server

`/api/x_acme_itsm_ai/catalog/v1 · sn_sc.CartJS / sn_sc.CatItem`

TOOL	WHAT IT DOES	TYPE
`search_catalog`	Find items the user is entitled to see	read
`get_item_form`	Variables, choices, mandatory fields	read
`validate_variables`	Check answers before ordering	read
`order_item`	`order_now` → REQ / RITM	confirm
`get_ritm_status`	Stage, approvals, fulfilment tasks	read
`list_my_requests`	Open RITMs for the user	read
`cancel_request`	Cancel an open RITM	confirm

Where the agent runs

Your cloud VPC recommended

Spring Boot API, coordinator, agents and MCP servers run as containers in your own AWS/Azure account. Calls to the instance are outbound HTTPS, locked down with OAuth plus the instance's IP address access control. Simplest to scale, and close to a hosted LLM.

On-prem data center

Same containers, same outbound HTTPS to `*.service-now.com`. Pick this when the LLM must be self-hosted on your own GPUs or data can't leave the network. No MID Server is needed for agent→instance calls. You only need one if ServiceNow must push events back in, for example a business rule that triggers the agent when a P1 is created.

Inside ServiceNow

The scoped app always runs on the ServiceNow cloud; that is fixed. You could also move the agents themselves in with Now Assist AI Agents, but then the coordinator, model choice and evaluation follow ServiceNow's runtime and licensing instead of your code. Check what your instance's release supports before going this way.

Design decisions

Authorization lives in the scoped app

Each Scripted REST resource has *Requires ACL authorization* on, guarded by a REST_Endpoint ACL and a scope role such as `x_acme_itsm_ai.agent_user`. Queries use `GlideRecordSecure`, not `GlideRecord`, so table and field ACLs still apply.

Act as the user, not as a service account

MCP servers pass the user's OAuth token, so `gs.getUserID()` inside the scripts is the real person. Catalog entitlements, ACLs and the audit trail all follow from that.

Cross-scope access

incident, problem and change_request are global tables. Grant read/write through Cross-Scope Access records, and keep the scoped app's own audit table for every tool call.

Writes need a human yes

Every *confirm* tool returns a preview first. The UI shows the exact record change and only then calls the tool with a confirmation id. Emergency changes never auto-submit.

Coordinator hand-offs

Incident → Problem when `find_similar` returns a cluster; Problem → Change when a fix needs deployment; Catalog → Incident when an order fails. Hand-off context lives in the session store, not in the prompt.

One MCP server per process

Each server is its own Spring Boot service (Spring AI MCP server starter), deployed and versioned separately, with its own rate limit against the instance.

Storage split

MongoDB for session history and inter-agent state; MySQL for the cost tracker, eval results and tool-call audit log.

Evaluation set

Replay a few hundred closed tickets: score routing accuracy, correct tool choice, and whether the agent's proposed assignment group matches what humans chose.

Guardrails

A prompt instruction is only a suggestion. A check in code or on the platform is a guarantee. Every guardrail that matters here is enforced in code or by ServiceNow.

LAYER	WHERE IT LIVES	WHAT IT STOPS
1. Input	Orchestrator API	Redacts PII, detects prompt injection in the user message, rejects off-topic requests.
2. Tool results	MCP servers / harness	Indirect prompt injection. Ticket descriptions and work notes are user-typed, so they are wrapped and treated as data, never instructions.
3. Tool access	Agent config	Least privilege. The Incident agent only sees Incident and CMDB tools, so it cannot call `create_change` even if tricked.
4. Tool arguments	MCP server (Spring validation)	Schema and enum validation, plus coded business rules: no emergency changes, no P1 resolve, reassign only within the user's groups.
5. Writes	Harness + UI	Two-step writes. The tool returns a preview and a `confirmation_id`; the commit happens only after the user clicks Confirm.
6. Platform	Scoped app	REST ACLs, `GlideRecordSecure`, data policies and business rules. The last line of defense if every layer above is fooled.
7. Output	Harness	Grounding check: every INC/CHG number, state or date in a reply must come from a tool result. Fields the user can't see are stripped.
8. Budget	Harness + cost tracker	Caps on loop iterations, tool calls, tokens and time per request.

Harness engineering

Agent loop

Spring AI ChatClient with tool callbacks, fixed stop conditions and a turn cap. A cheap model does intent routing; stronger models handle domain reasoning and change-risk analysis.

Tool design

Tools shaped around tasks, returning only the fields an agent needs. Descriptions say when to use each tool, and errors say what to do next ("call lookup_group first").

Context engineering

The session store holds structured state (current record, CI, user, pending confirmations). Long ticket histories are summarized, and hand-offs pass typed JSON.

Reliability

Idempotency keys on every create, so a retry never makes a duplicate INC. ServiceNow writes are never retried blindly. Timeouts are set per MCP server.

Observability

One OpenTelemetry trace per request: prompt, each tool call with arguments and result, latency, tokens and cost. Traces go to MongoDB; the audit summary goes to MySQL.

Evals

Offline replay of closed tickets, plus an adversarial set (injected instructions, out-of-scope records, emergency-change attempts). CI blocks any prompt or tool change that lowers scores.

One request through every layer

1. **Input.** "Close INC0012345, the fix is deployed." PII is redacted and the coordinator routes to the Incident agent.
2. **Read.** `get_incident` returns work notes wrapped as data. A planted instruction in one note is flagged and ignored.
3. **Propose.** The agent calls `resolve_incident`. The MCP server sees a P1 and blocks it by code rule, so the agent says P1s need a manual resolve.
4. **Confirm.** On a P3, the agent gets a preview and a `confirmation_id`. The Scripted REST API runs only after the user clicks Confirm.
5. **Platform.** `GlideRecordSecure` checks that this user can write to this incident.
6. **Record.** The trace, cost and audit row are saved.